MLOPS WITH CLOUD PLATFORMS

© 2011-25 National University of Singapore. All Rights Reserved.

Topics

- Introduction to MLOps
- Pipeline design
- Monitoring and Logging
- **MLOps with Cloud platforms**

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Popular cloud platforms

- Amazon Web Service (AWS)
- Azure
- Google Cloud Platform (GCP)

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Setting up an S3 bucket**
  - Store ML runs by syncing local mlruns folder
  - Centralized Model Registry
- **Building and Pushing the MLFlow Docker Container**
  - Docker container is built and pushed to **AWS Elastic Container Registry (ECR)** using the mlflow sagemaker build-and-push-container command

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Deploying model to AWS SageMaker**
  - Get the image from ECR by specifying the image url
  - Get the model information from S3 bucket by specifying the model uri
  - deployment is initiated using the **mlflow.sagemaker.deploy function** with the mode set to mfs.DEPLOYMENT_MODE_CREATE
  - status of the SageMaker endpoint can be monitored in the AWS console, changing from "creating" to **"InService"** upon successful deployment

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Making Prediction**
  - predictions are made using **boto3**, the AWS SDK for Python
  - **batch querying** is necessary for large datasets as there is a limitation on number of samples per request
- **Switch model**
  - mlflow.sagemaker.deploy again, but setting the mode to **mfs.DEPLOYMENT_MODE_REPLACE** and providing the new_model_uri
  - Don’t need to delete and recreate

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Remove deployed model**
  - **mlflow.sagemaker.delete function** is used to remove the endpoint and the deployed model, requiring the app_name and region
  - Remove deployed model when you are not in use. It costs $$$!

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Install the azureml-sdk in your Python environment**
- **Create an Azure workspace**
- **Setting up an BLOB**
  - Store ML runs by syncing local mlruns folder
  - Centralized Model Registry
- **Building and Pushing the MLFlow Docker Container**
  - Deploy to Azure Container Instances (ACI)

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Deploying model to Azure**
  - ACI webservice specifically for debugging or testing models in a development stage
  - Deploy an ACI webservice instance based on the created model image using Webservice.deploy_from_image
  - MLFlow provides built-in support to deploy models directly to production using a container instance

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Making Prediction**
  - predictions are made by querying the production endpoint's scoring_uri with appropriately scaled data
  - **batch querying** is necessary for large datasets as there is a limitation on number of samples per request
- **Switch model**
  - MLFlow does not provide explicit functionality to switch models on an existing Azure service
  - Requires to delete and recreate

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- MLFlow does not provide explicit support for deploying in Google Cloud
- Install Google Cloud SDK
- **Setting up an Google Storage bucket**
  - To store the contents of your mlruns folders
- **VM (Setup)**
  - To host the model
  - Installing **Conda** (essential for MLFlow to reconstruct environments) and **MLFlow** within the VM

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- **Deploying model to GCP**
  - model is served by executing the mlflow models serve command directly on the VM
- **Making Prediction**
  - HTTP POST requests with JSON data (using application/json) to the VM's external IP address on port 5000(for example)
  - Batch querying is implemented for larger datasets, breaking them into smaller samples due to request limits

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- **Switch model**
  - Doesn’t support swtich directly.
  - Cancel the current ***mlflow models serve*** command
  - Re-run the ***mlflow models serve command*** is re-run with the new model's URI (run ID)
- **Remove model**
  - Cancel the current ***mlflow models serve*** command

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Comparison Table

Category	AWS	Azure	Google Cloud
ML Platform	SageMaker	Azure Machine Learning	Vertex AI
Data Storage	S3	Blob Storage	Cloud Storage
Data Pipeline	Glue, Data Pipeline, Step Functions	Data Factory	Dataflow, Composer (Airflow)
Training Compute	SageMaker Training, EC2	Azure ML Compute	Vertex Training / GKE
Experiment Tracking	SageMaker Experiments	Azure ML Run History	Vertex ML Metadata
Model Registry	SageMaker Model Registry	Azure Model Registry	Vertex Model Registry
Deployment	SageMaker Endpoint / Lambda	AKS / Azure Functions	Vertex Prediction / Cloud Run
CI/CD	CodePipeline + CodeBuild	Azure DevOps / GitHub Actions	Cloud Build + Cloud Deploy
Monitoring	SageMaker Model Monitor, CloudWatch	Azure Monitor + ML Monitor	Vertex Monitoring, Cloud Monitoring
Drift Detection	SageMaker Model Monitor	Custom with Azure Monitor	Vertex Model Monitoring
Security	IAM, KMS, PrivateLink	RBAC, Key Vault	IAM, VPC, Data Loss Prevention

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Best Practices

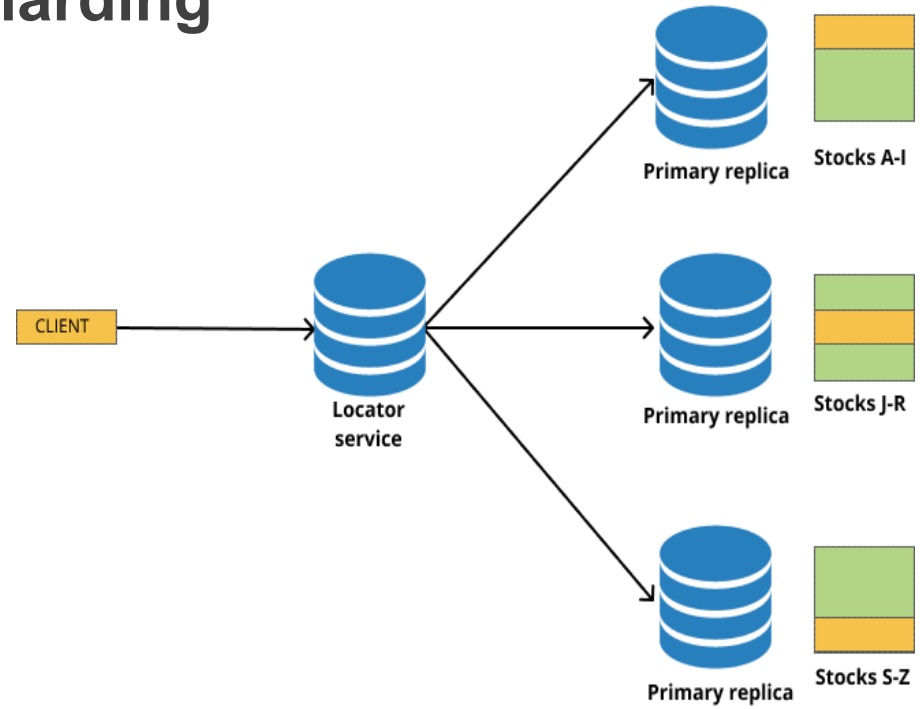
- Use versioning for datasets, models, and code.
- Implement access control roles with fine-grained access.
- Schedule model retraining via Serverless such as AWS Step Function to be more cost effective.
- Store metrics and logs in a log repository and storage such as S2 or Blob storage for audits.
- Use multi-account strategy for dev/test/prod segregation.

© 2011-24 National University of Singapore. All Rights Reserved.

Scale the ML System – Data Scaling

Data scaling using data sharding

- A shard is a subset of data
- One master per shard
- Partitioning of data across several machines
- Improves both write and read performance
- Locator service to let client know which master is responsible for which shard



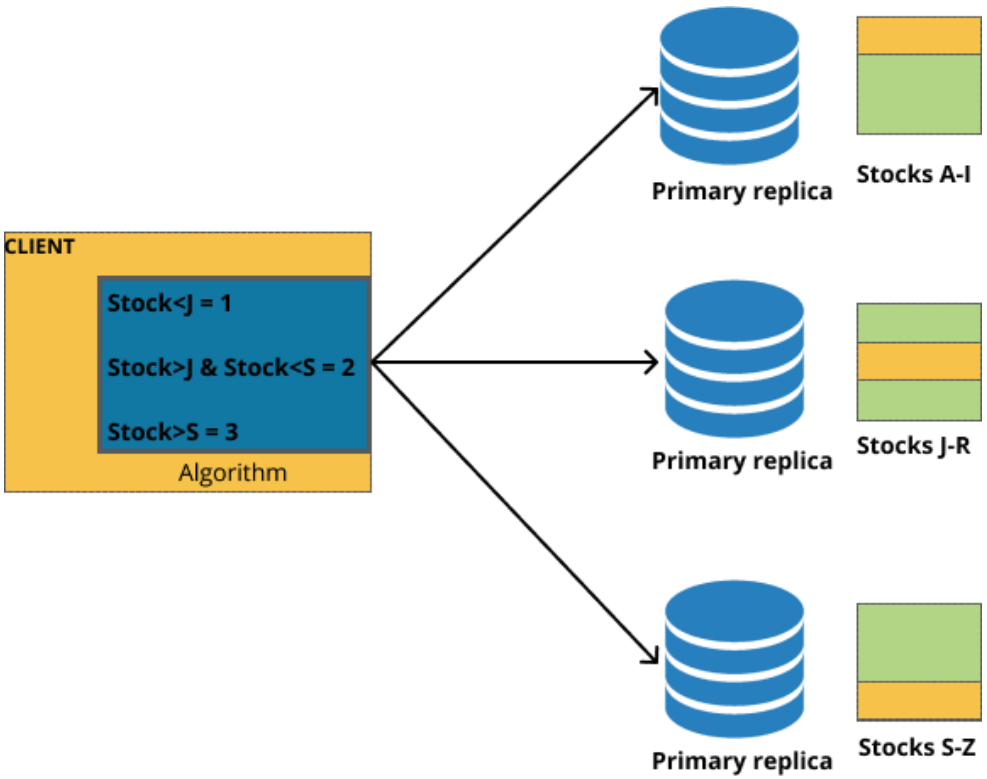
Problem ???

© 2011-24 National University of Singapore. All Rights Reserved.

Scale the ML System – Data Scaling

Data sharding using algorithm

- No need of locator service
- Algorithm resides in Client
- Fixed sharding key may not distribute data properly
- Hash based partitioning would distribute more evenly
- Just like a regular HashMap
- If shard changes, masters publish events



© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Scale the ML System – Pipeline Scaling

Saga Design Pattern

- **Long-running transaction pattern** used to **maintain consistency** across microservices or distributed systems **without 2PC (two-phase commit)**
- breaks a distributed transaction into a **series of smaller, isolated transactions (called "saga steps")**

© 2011-24 National University of Singapore. All Rights Reserved.

Scale the ML System – Pipeline Scaling

Saga Design Pattern Types

- **Orchestrated Saga**: A central coordinator handles step-by-step logic (recommended for ML)

- **Choreographed Saga:** Each service emits events that trigger the next step (used in event-driven ML pipelines)

© 2011-24 National University of Singapore. All Rights Reserved.

Scale the ML System – Pipeline Scaling

Apply Saga Design Pattern in ML Pipelines

Step	Saga Step	Compensation
1. Data Ingest	Store data from upstream	Delete invalid record
2. Preprocess	Clean, normalize data	Revert changes in feature store
3. Train Model	Use cleaned data	Delete model artifacts
4. Register Model	Add to model registry	Remove model version
5. Deploy Model	Expose endpoint	Rollback to previous version
6. Notify	Send completion alert	Send failure alert

Each of these can be:
A separate **microservice or function**
Managed using **orchestration tools** (like **Azure Durable Functions**, **AWS Step Functions**)

© 2011-24 National University of Singapore. All Rights Reserved.

Saga Design Pattern

Benefits

- **Resilient:** Faults in training or deployment won't break the whole system
- **Auditable:** Each step is logged
- **Compensable:** Easier rollback on failures
- **Modular:** Encourages reusable, containerized ML steps

Hence, the system can be safely scaled !

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Managing resources and cost optimization

- **Right Infrastructure**
  - ❖ Auto-scaling Kubernetes pods for on-demand workloads
  - ❖ Serverless (e.g. AWS Lambda, Azure Functions) for bursty monitoring tasks
- **Job Scheduling & Priority Queues**
  - ❖ Schedule heavy jobs (e.g., retraining) in **off-peak hours.**
  - ❖ Use on-demand charging and serverless model like AWS Lambda, Azure Functions for scheduled jobs.

© 2011-24 National University of Singapore. All Rights Reserved.

Managing resources and cost optimization

- **Limit Resource Quotas**
  - Define **per-team and per-project quotas** (CPU, GPU, storage)
- **Tracking**
  - Prometheus + Grafana - to track the system and infrastructure usage, set alert etc
  - Kubecost – real time cost monitoring of K8s
  - Predict the infra cost using tools like **Terraform Cost Estimation**

© 2011-24 National University of Singapore. All Rights Reserved.

Managing resources and cost optimization

- **Resource saving model serving**
  - ❖ Consider hosting infrequent requested models in serverless solution such as AWS Lambda/Azure Function or AWS SageMaker Serverless Inference/ Azure ML Serverless Endpoints

Example: AWS SageMaker Serverless
Source: https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Managing resources and cost optimization

- **Deployment**
  - Perform A/B testing to reduce unnecessary deployment
- **Storage**
  - Archive unused models to cold storage (e.g., AWS Glacier, Azure Archive)
  - Clean old versions using lifecycle policies

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- MLFlow and deployment with 3 most popular cloud platforms and best practices
- Scale the ML System by data sharding
- Adopt Saga Design pattern to scale MLOps pipelines.
- Ways to manage resources and optimize cost

© 2011-24 National University of Singapore. All Rights Reserved.